Robotic Path Planning Software Design Documentation

Bruun, E. Flores, J. Kumar, V.

December 6, 2019

1 Introduction

This software is designed to solve the classic problem of robotic path planning. Given a start and end point, a path must be found that does not collide with a set of obstacles. The space for which the path can be found is not discretized, so it does not have existing nodes for the robot to travel between. Due to this, a graph of vertices and edges must be created. In addition to generating the graph, determining which is the shortest path must also be solved.

Currently, the path planning problem is one of the most researched topics in autonomous robotics. There are many path planning algorithms that address this problem. Many algorithms employ a grid based search, but we aim to solve the path planning problem with an interval based search to cover the entirety of the free space instead of a grid.

This software focuses on implementing Rapidly-exploring random trees (RRT) algorithms. RRT is a common option that both creates a graph and finds a path, but the path will not necessarily be optimal. RRT*, popularized by Dr. Karaman and Dr. Frazzoli, is an optimized modified algorithm that aims to achieve a shortest path, whether by distance or other metrics. Although these are the algorithms implemented, other solvers could be added and utilized in the future.

This software will be developed such that it can be implemented in both 2D and 3D. The ultimate goal is to utilize this software for the design of structures such as bridges that span a certain distance while avoiding specified obstacles in the way. The intended audience is structural designers working on preliminary designs. The actual structural analysis of the resulting structure is out of the scope of this project, but could easily be added as a feature in the future.

2 Design

2.1 UML Class Diagram

Top level schematic and description

2.1.1 Input

Submodule A schematic and description

2.1.2 Solver

Submodule B schematic and description

2.1.3 Output

Submodule C schematic and description

3 Algorithms

3.1 RRT

Add description and pseudocode

3.2 RRT*

Add description and pseudocode

3.3 Testing

State the approach to verify the functional correctness of the modules.

4 Results

4.1 RRT

4.2 RRT*

5 Profiling

Speed up attempt, what was slow etc.

6 Conclusion

Comparison and conclusions, how it was working together

7 References